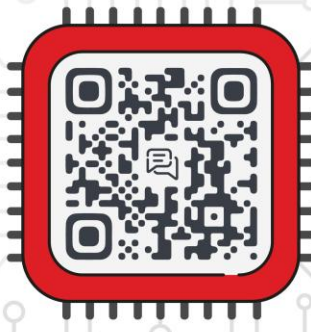




6

15

1800 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

أعطال التوارد

Hazards Pipelining

عملي مشترك



RB Informatics; 2025/12/13

بنیان الحواسيب 2

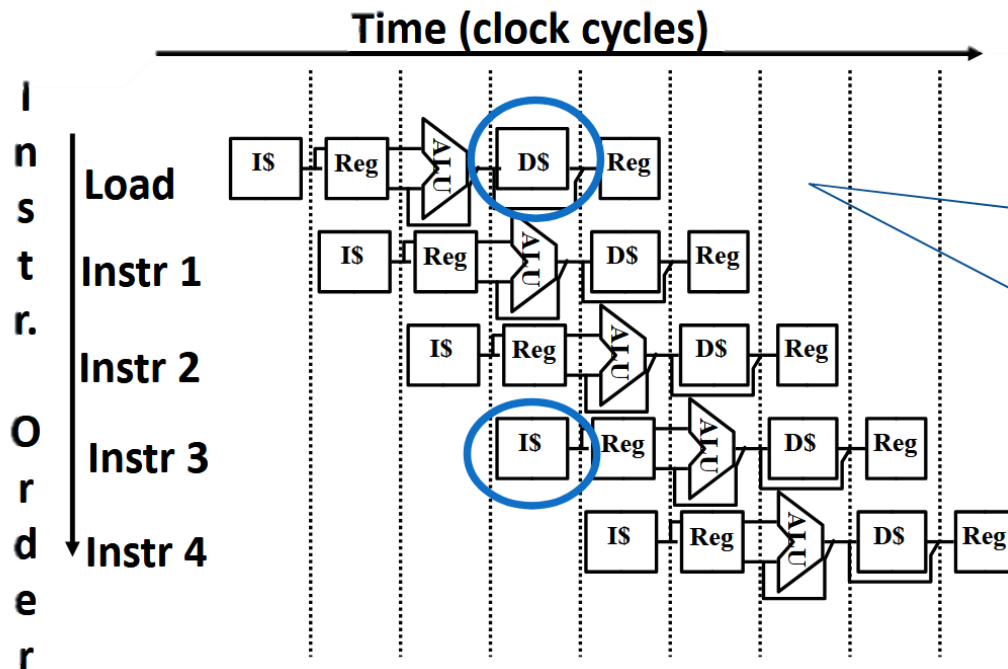
أنواع الأعطال

1. أعطال بنيوية Structural Hazard:

تحدث عندما أكثر من تعليمة تستخدم نفس المورد (Resource) بالمعالج في نفس اللحظة أي يوجد تزامن على Hardware-

مثال:

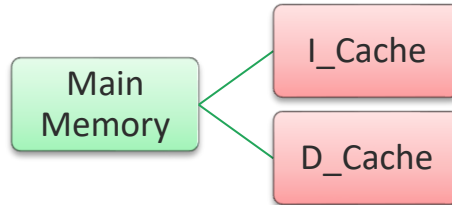
إذا كان لدينا ذاكرة واحدة للقراءة والكتابة وتعليمة تقوم بعمل Fetch من الذاكرة وتعليمة أخرى بنفس اللحظة تقوم بعمل Load/Store سيتنافسون على نفس الذاكرة أي سيحدث Hazard وتسمى بـ Resource conflict



قراءة الذاكرة مرتين في نفس اللحظة (نفس الدور)!!

الحلول الممكنة: يوجد لدينا حلان:

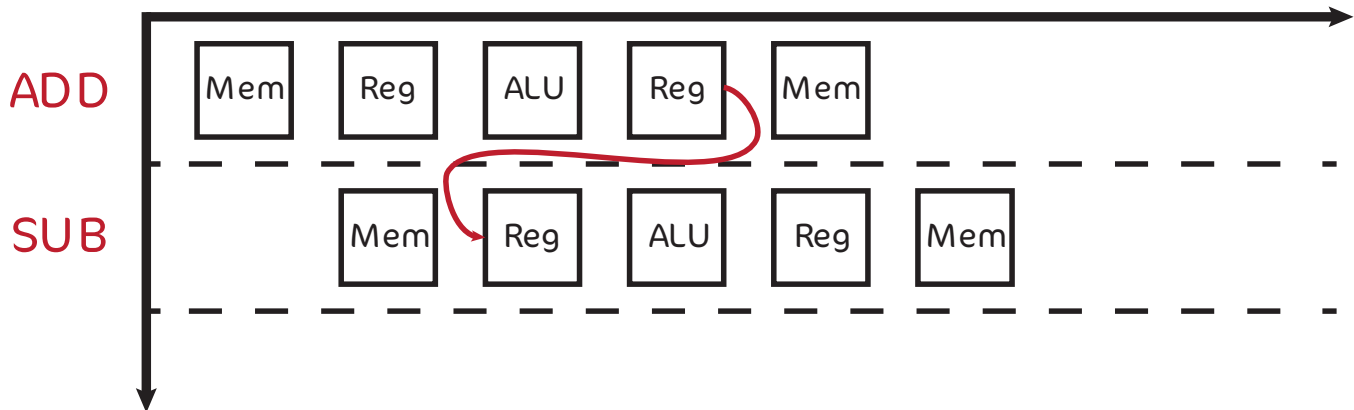
1. الحل الأول هو عن طريق التأخير الزمني stall ويمكننا استخدامه لأي مشكلة تواجهنا، لكنه يقلل أداء المعالج.
2. تقسيم الذاكرة إلى قسمين، ذاكرة للتعليمات (I_Cache) وذاكرة للمعطيات (D_cache) وبذلك يمكننا قراءة تعليمة وقراءة data في نفس اللحظة.

2. أعطال المعطيات Data Hazard:

يحدث عندما يكون لدينا تعليمة تعتمد على نتيجة تعليمة قبلها لكل النتيجة لم تكتب على الذاكرة بعد

مثال:

1. `ADD R1, R2, R3`
2. `SUB R4, R1, R5`



نلاحظ قيمة R1 الجديدة لم تكتب بعد على register.

الحلول الممكنة: يوجد لدينا ثلاث حلول:

1. التأخير الزمني Stall / إضافة فقاعات Bubbles: حل عتادي يحتاج إلى دارة كشف الأعطال (Hazard Detection Unit)
2. إضافة دارة إحالة Bypassing/Forwarding Unit للحصول على المعطيات دون انتظار الحالة النهائية على الوجهة
3. إعادة ترتيب المعالج عن طريق المترجم (Compiler) بحيث تتم المباشرة بين التعليمات المترابطة وإضافة NOP عند الحاجة (حل برمجي)

ملاحظة:

في معالج MIPS:

NOP = 'No Operation' = 00...0 (32 Bits) = `sll $0, $0, 0` #Shift Left Logical

3. أعطال التحكم Control Hazard:

تحدث عندما تكون التعليمة عبارة عن قفز أو تفرّع (J, Beq, Bne) والمعالج لا يعلم أين يكمل التنفيذ.

مثال:

BEQ R1, R2, Label

المعالج يحتاج أن يعرف هل $R1 = R2$ ؟

- إذا نعم - jumb <

- إذا لا - < يكمل

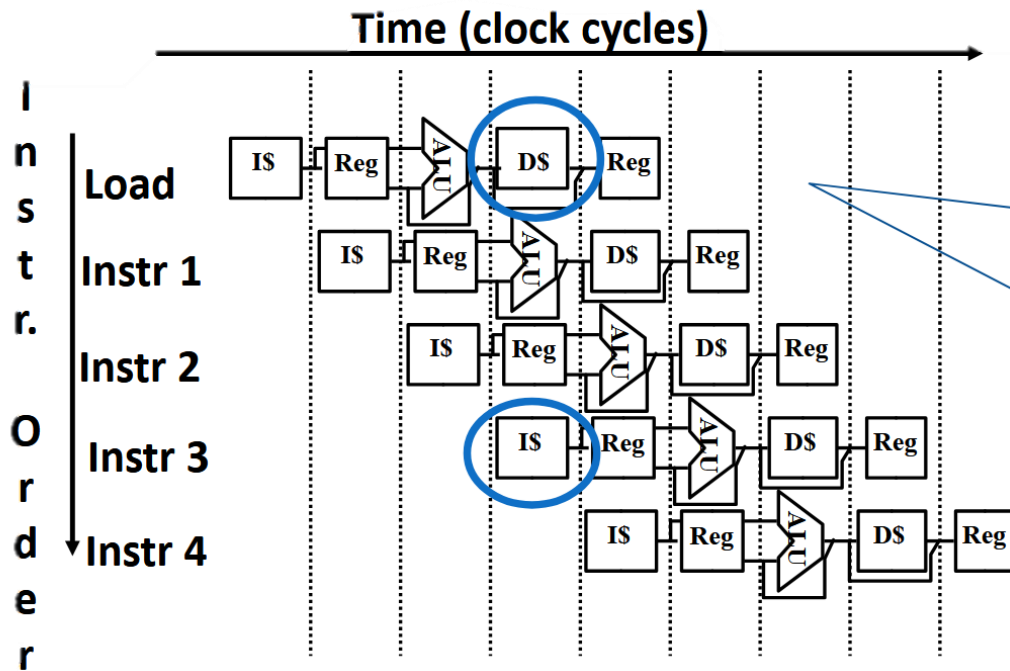
وخلال انتظار النتائج سيكون قد قرأ تعليمات خطأ بالتالي حصل Hazard واسمه Branch Hazard

الحل:

إما Branch Predication أو Delay Slot

أمثلة عن الأعطال البنيوية:

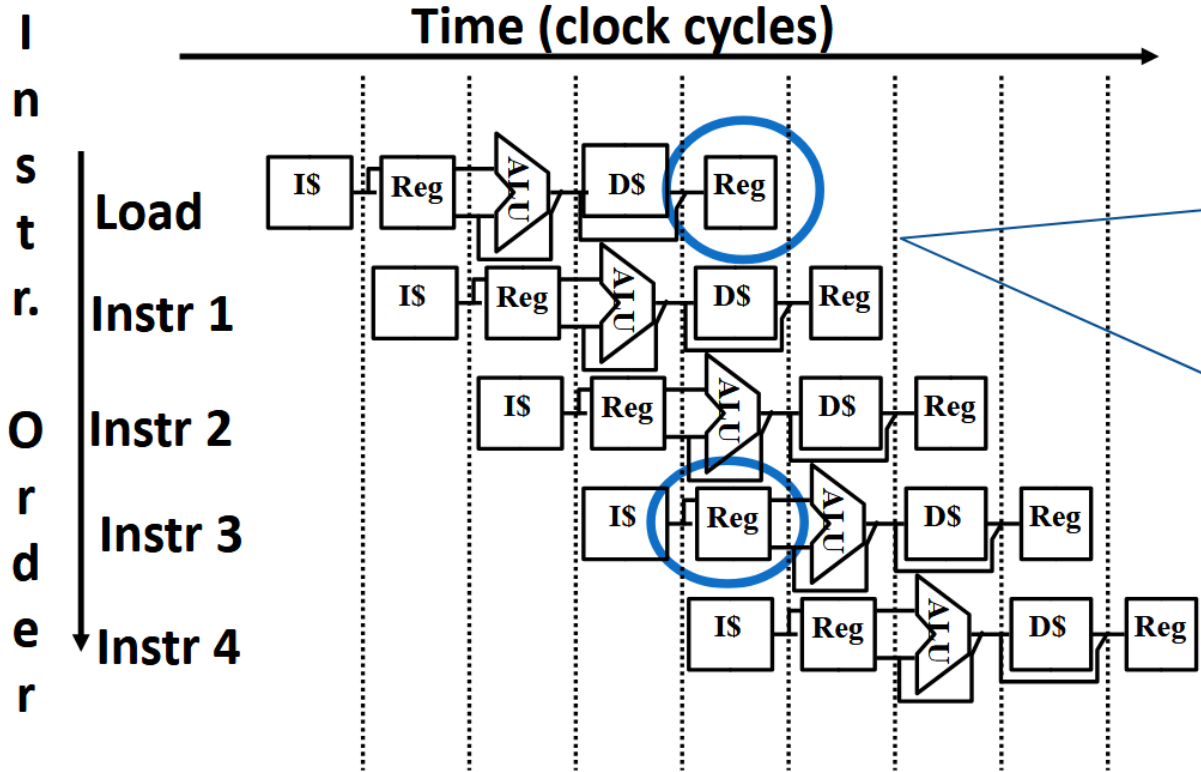
المثال الأول:

الشرح:

ليكن لدينا مجموعة التعليمات المتواردة في الشكل، أول تعليمة هي Load وبعدها أربع تعليمات أخرى، عند دور الساعة الرابع تعليمة load في مرحلة D\$ أي في الذاكرة الخارجية Cash Memory وفي نفس دور الساعة التعليمة الثالثة Instr 3 أيضا تقوم بالقراءة من الذاكرة الخارجية I\$ أي يوجد لدينا تعليمتي قراءة في نفس الوقت وبالتالي حدث لنا تنازع على الموارد

إما أن نؤخر التعليمات الثالثة دور واحد أو أن نضيف ذاكرة Cash أخرى

المثال الثاني:



الشرح:

ليكن لدينا في الرسم تعليمات Load وأربع تعليمات أخرى، في دور الساعة الرابع لدينا تعليمات Load تقوم بالكتابة على Register File والتعليمات الثالثة أيضاً تقوم بالقراءة من Register File، وبالتالي أصبح لدينا تنازع على الموارد

الحل:

مر معنا خلال المحاضرات السابقة أن زمن الدور الأصغري في معالج ال pipe line هو 4 ns وزمن القراءة والكتابة في Register File هو 2 ns وبالتالي يمكن القيام بعملية قراءة وعملية كتابة في نفس الدور أي نقوم بالكتابة على السجل أثناء النصف الأول من دور الساعة، ثم نقوم بالقراءة من السجل في النصف الثاني من دور الساعة، وبالتالي يمكن القراءة والكتابة في السجل في دور الساعة نفسه.

ملاحظة:

نقوم بعملية الكتابة قبل عملية القراءة، لأنني قد أحتاج إلى قراءة نفس القيمة التي أريد كتابتها، فإذا قمت بعملية القراءة قبل عملية الكتابة فسوف أقرأ القيمة القديمة الموجودة في السجل لأن القيمة الجديدة لم تكتب على السجل بعد.

لدينا معالجان يتم تنفيذ برنامج معين على كل منهما، المعالج الأول A (مثالي / Ideal / without hazard) يحتوي على ذاكرة منفصلة (Split Cache) فصل الخابية، لكل من التعليمات والبيانات، فيه $CPI_A = 1$ ، زمن دور الساعة T

المعالج الثاني B (ذاكرة موحدة / Unified Memory / with hazard) تردد هذا المعالج أسرع بـ 1.25 مرة من المعالج المثالي A، يحتوي على منفذ واحد فقط للذاكرة، فإذا علمت أن تعليمات الذاكرة (Load/Store) تشكل 30% من البرنامج المنفذ

المطلوب: أي المعالجين أسرع في تنفيذ البرنامج بفرض عدم وجود أعطال من الأنواع الأخرى؟

الحل:

$$CPUtime_A = IC \times CPI \times T = IC \times 1 \times T$$

$$CPUtime_B = IC \times CPI_B \times T_B$$

$$T_B = \frac{T}{1.25} = 0.8T$$

$$CPI_B = (0.70 \times 1 + 0.30 \times 2)$$

$$CPI_B = 0.70 + 0.60 = 1.30$$

$$CPUtime_B = IC \times 1.30 \times 0.8T = IC \times 1.04 \times T$$

$$\frac{CPUtime_B}{CPUtime_A} = \frac{IC \times 1.04 \times T}{IC \times 1 \times T} = 1.04$$

الملاحظات:

- نجد أن زمن تنفيذ المعالج B ذي الأعطال البنيوية بسبب الذاكرة الموحدة أطول من زمن تنفيذ المعالج المثالي (رغم أن تردده أعلى بنسبة 25%) وبالتالي فإن المعالج المثالي بدون الأعطال أسرع بـ 1.04 مرة من المعالج ذي الأعطال بسبب الذاكرة
 - المعالج الأسرع A والأبطأ B
 - ويوجد لدينا قانون لحساب CPI_B مذكور بالمراجع تستطيع استخدامه
- $$CPI_B = \text{Base CPI} + (\text{Rate} * \text{Penalty}) = 1 + (0.30 * 1) = 1 + 0.30 = 1.30$$
- ليس دائما التردد الأعلى يعطي معالج أسرع أو أفضل

وظيفة:

أعد حل المسألة بفرض كانت نسبة تعليمات الذاكرة هي 25%، ثم بفرض نسبتها 10%.

الحل:

عند النسبة 25 يكون الزمنين متساويين، وعند النسبة 10 يكون المعالج B أسرع والمعالج A أبطأ

أمثلة عن أعطال المعطيات Data Hazard

مثال:

بفرض أن المحتوى الابتدائي لأي سجل هو رقم السجل $10 \times$ (مثلا محتوى السجل \$3 هو 30) ما هو المحتوى النهائي للسجلات في نهاية تنفيذ كل تعليمة من التعليمات التالية على معالج متعدد الأدوار (Multi Cycle) وعلى معالج متوارد ؟PL

الحل:

1. على MC

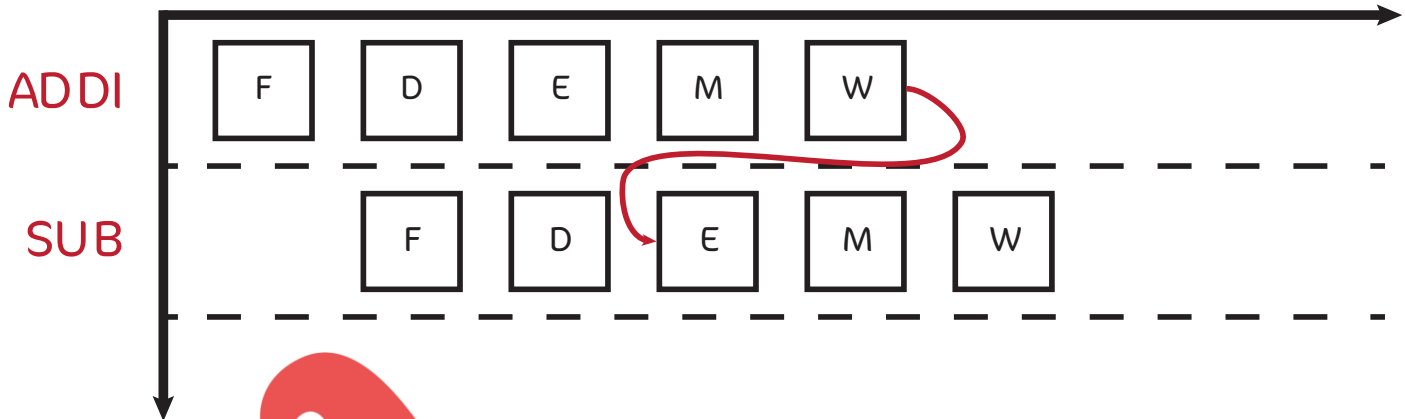
1 - Addi \$9, \$3, 5 # \$9 = \$3 + 5
 F D E W = 30 + 5 = 35
 2 - Sub \$5, \$6, \$9 # \$5 = \$6 - \$9
 F D E W = 60 - 35 = 25

2. على PL

1 - Addi \$9, \$3, 5 # \$9 = \$3 + 5
 2 - Sub \$5, \$6, \$9 # \$5 = \$6 - \$9

يتم حفظ الناتج 35 في R.F بالدور 5 بمرحلة (Write Back)

ستكون قيمة \$9 غير محدثة بعد، أي ستكون 90 بدل 35

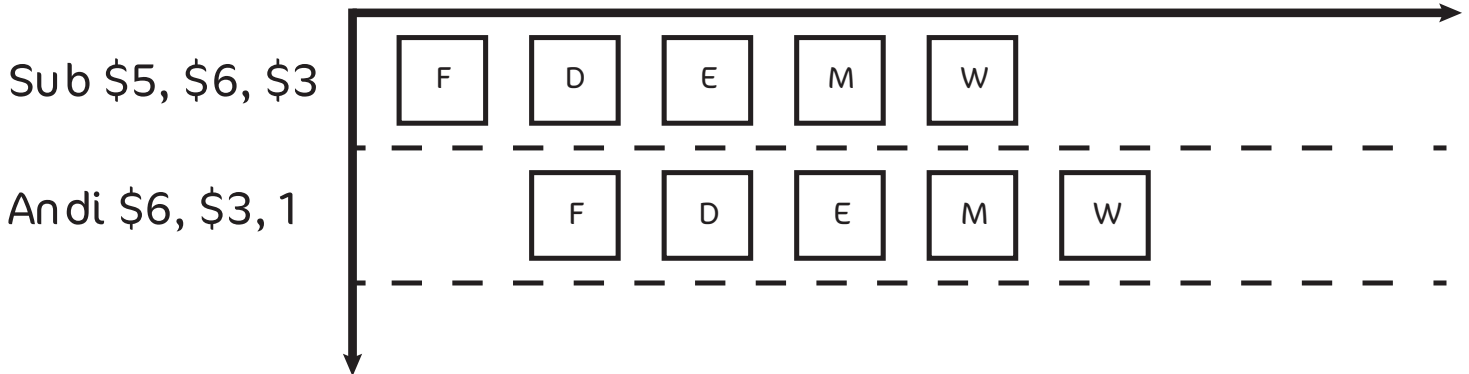


اللهم لا سهل إلا ما جعلته سهلا،
 وأنت تجعل الحزن إذا شئت سهلا

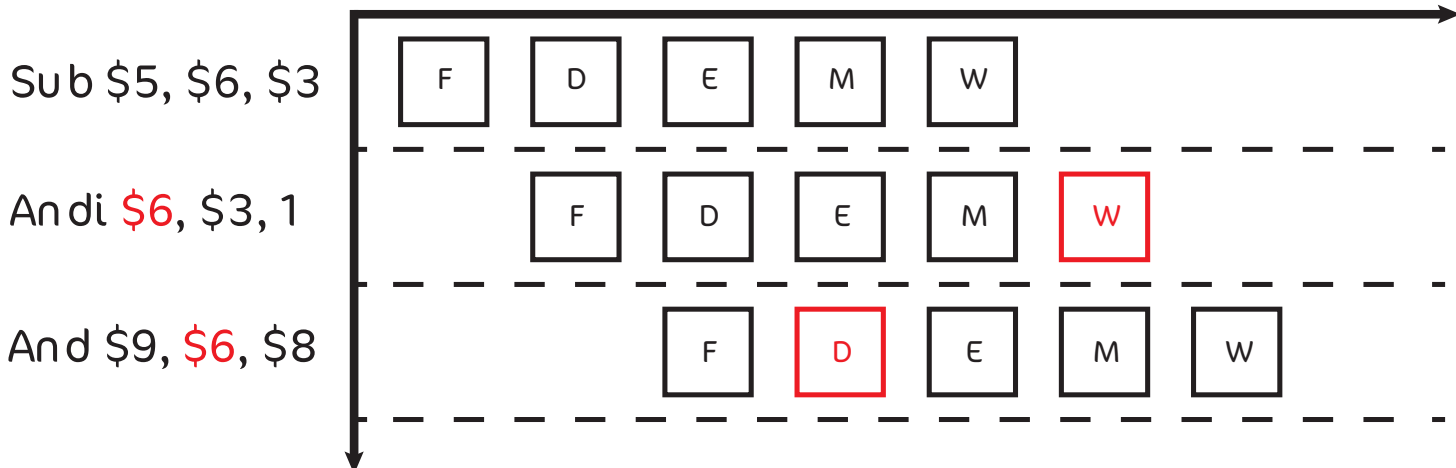
لدينا البرنامج التالي ينفذ على معالج MIPS متوارد، حدد ترابطات المعطيات الموجودة في البرنامج، ثم مثل أدوار التنفيذ باستخدام طريقة التأخير الزمني Stall (إضافة فقاعات Bubble) لمعالجة أعطال المعطيات بفرض عدم وجود أعطال بنيوية، وبفرض عدم إمكانية النفاذ المزدوج لملف السجلات (عدم إمكانية الكتابة والقراءة عليه بنفس الدور) (باعتبار أن قراءة السجلات تتم في المرحلة D)

التعليمات:

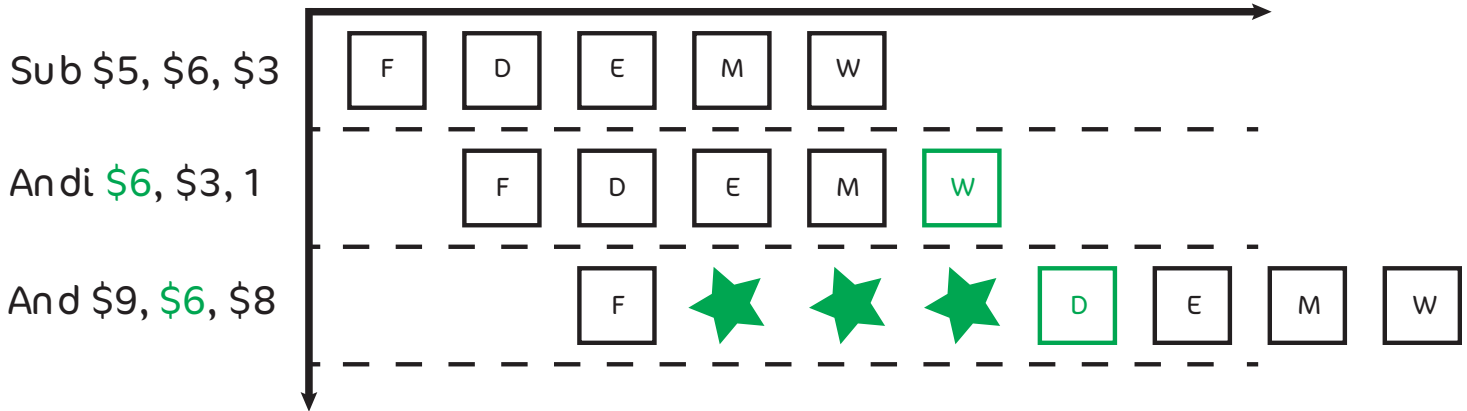
1. Sub \$5, \$6, \$3
2. Andi \$6, \$3, 1
3. And \$9, \$6, \$8
4. Add \$5, \$2, \$7
5. Lw \$7, 0(\$9)
6. Sw \$5, 4(\$7)
7. Or \$3, \$2, \$7

الحل:

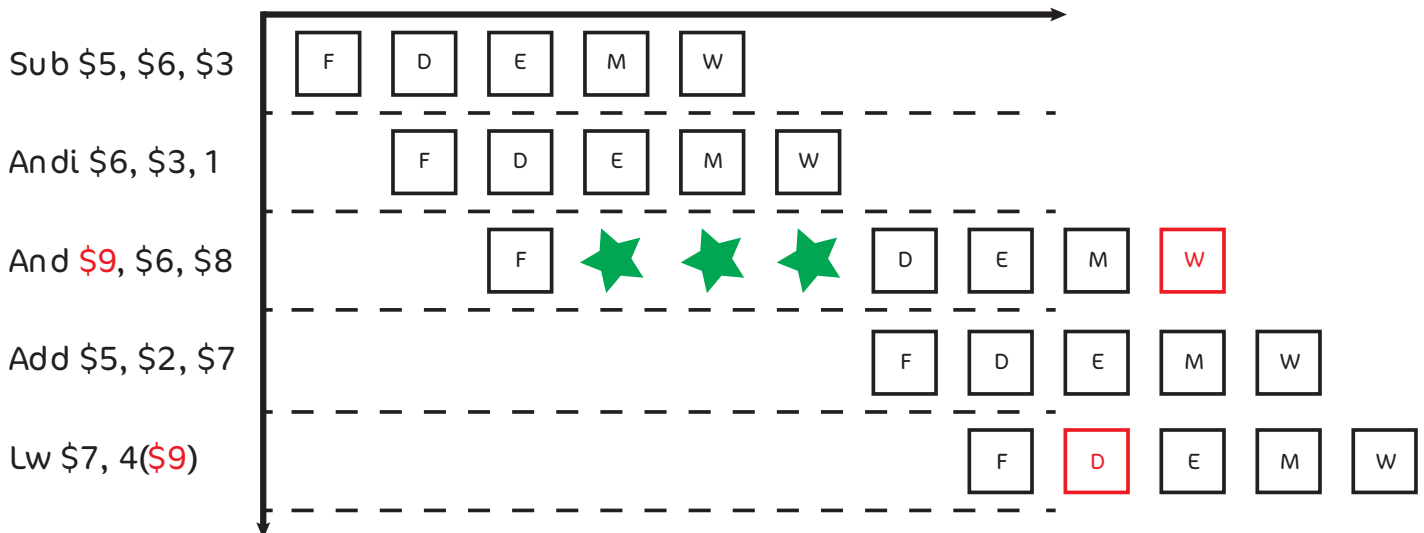
- التعليمة الأولى: تتم مراحلها بشكل صحيح بخمس أدوار (لا يوجد أعطال)
- التعليمة الثانية: لدينا كتابة على السجل \$6 بدور الساعة 6 (لا يوجد أعطال)



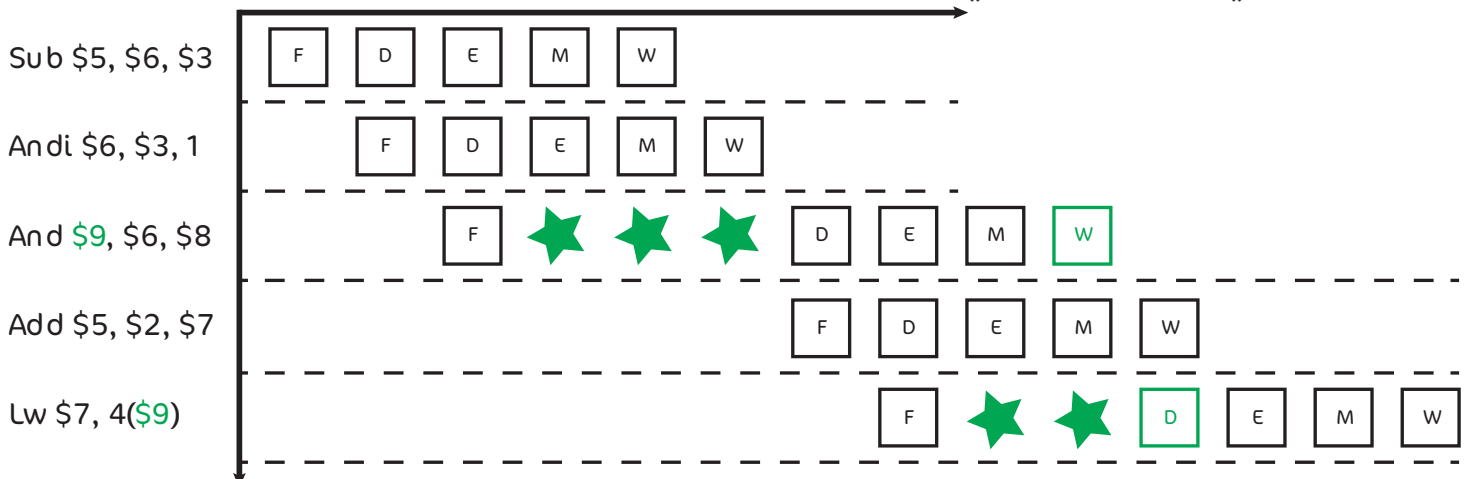
- التعليمية الثالثة: في دور الساعة الرابع D تحتاج إلى قراءة محتوى السجل السادس من الذاكرة والتعليمية الثانية لك تقوم بكتابة محتوى السجل \$6 بعد، هنا نحتاج Stall ثلاث أدوار حتى تنتهي التعليمية السابقة من كتابة محتوى السجل 6، كالآتي:



- التعليمية الرابعة: يوجد فيها كتابة في السجل 5 بالدور 11



- التعليمية الخامسة: لدينا كتابة على السجل السابع وقراءة من السجل التاسع دخلت مرحلة Fetch في الدور الثامن، تحتاج إلى D أي قراءة السجل التاسع بعدها لكن في التعليمية الثالثة لم تكتب قيمة السجل التاسع بعد لذلك سيحدث Stall دورين من الساعة حتى تستطيع قراءة القيمة الصحيحة للسجل التاسع، وبالتالي يصبح الجدول كالآتي:



ملاحظة هامة جداً:

LW rt, im (rs)
LW rt <- M[im+rs]

SW rt, im (rs)
SW rt -> M[im+rs]

مثال: SW \$5, 4(\$3)

السجل \$3 قراءة، والسجل \$5 قراءة

والكتابة تحدث في الذاكرة وليس في السجل

ولدينا أيضاً تعليمات Branch

Beq \$3, \$2, Label

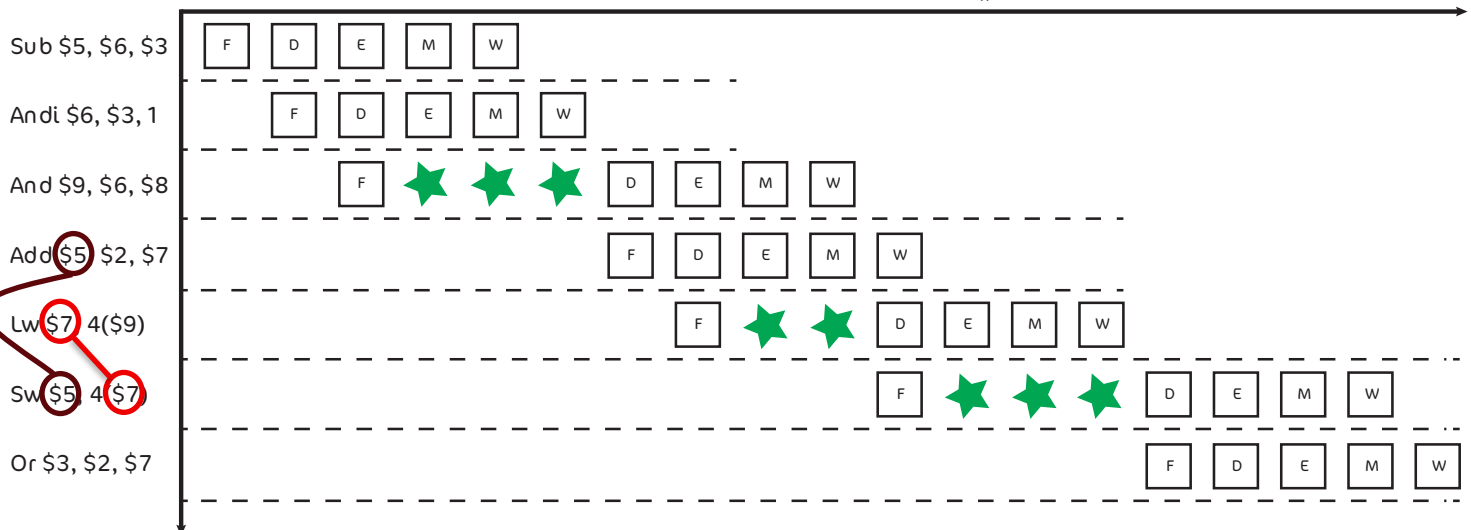
Bne \$5, \$4, Label

قراءة فقط من السجلات لا يوجد كتابة عليها

ملاحظة 2:

تحت الفقاعة لا يوجد مراحل لأنه أصبح الدور كامل في مرحلة Stall

- التعليمة السادسة: يوجد لدينا في هذه التعليمة قراءة من السجل الخامس ولا نستطيع عمل Fetch في الدورين التاسع والعاشر حتى تقوم التعليمة الرابعة بكتابة محتوى السجل الخامس أولاً لذا يصبح لدينا تأخير دورين ثم لا ننتقل إلى Decode حتى تتم كتابة محتوى \$7 من قبل التعليمة الخامسة حتى تتم قراءته بشكل صحيح، وبالتالي يصبح لدينا تأخير ثلاث أدوار أيضاً
- التعليمة السابعة لا يوجد فيها أي تأخير



$$CPI = \frac{[5 + (IC - 1) + Stalls]}{IC} = \frac{[5 + 6 + 8]}{7} = \frac{19}{7} = 2.7$$

تمرين:

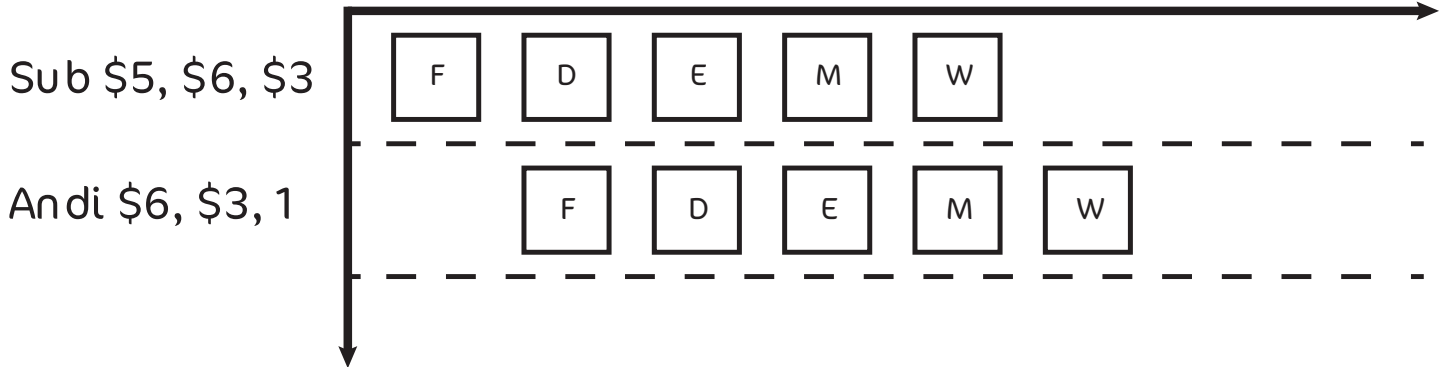
لدينا البرنامج التالي ينفذ على معالج MIPS متوارد، حدد ترابطات المعطيات الموجودة في البرنامج، ثم مثل أدوار التنفيذ باستخدام طريقة التأخير الزمني Stall (إضافة فقاعات Bubble) لمعالجة أعطال المعطيات بفرض عدم وجود أعطال بنيوية، وبفرض إمكانية النفاذ المزدوج لملف السجلات (إمكانية الكتابة والقراءة عليه بنفس الدور) (باعتبار أن قراءة السجلات تتم في المرحلة D)
التعليمات:

1. Sub \$5, \$6, \$3
2. Andi \$6, \$3, 1
3. And \$9, \$6, \$8
4. Add \$5, \$2, \$7
5. Lw \$7, 0(\$9)
6. Sw \$5, 4(\$7)
7. Or \$3, \$2, \$7

ملاحظة:

النفاذ المزدوج للذاكرة، فقط لمرحلتي Write Back مع Decode

الحل:



- التعليمة الأولى: تتم ضمن مراحلها بشكل متتالي
- التعليمة الثانية: يوجد فيها بالدور السادس تعليمة كتابة في السجل 6



Sub \$5, \$6, \$3

F D E M W

Andi \$6, \$3, 1

F D E M W

And \$9, \$6, \$8

F D E M W

- التعليمة الثالثة: يوجد فيها كتابة على السجل التاسع وقراءة من السجل السادس لذلك سنقوم بتأخير مرحلة Decode دورين تتم قراءتها في الدور السادس، ويمكننا القيام بذلك لأن الذاكرة ذات نفاذ مزدوج أي أستطيع القراءة والكتابة في نفس الدور، وبالتالي يصبح الجدول كالآتي:

Sub \$5, \$6, \$3

F D E M W

Andi \$6, \$3, 1

F D E M W

And \$9, \$6, \$8

F D E M W

- التعليمة الرابعة: لدينا كتابة على السجل الخامس

- التعليمة الخامسة: يوجد قراءة من السجل 9 وكتابة على السجل 7 لذلك نقوم بتأخير عملية ال D دور واحد حتى يتم كتابة محتوى السجل التاسع.

في الدور التاسع يمكننا قراءة وكتابة معاً دون حدوث مشاكل لأن الذاكرة ذات نفاذ مزدوج

Sub \$5, \$6, \$3

F D E M W

Andi \$6, \$3, 1

F D E M W

And \$9, \$6, \$8

F D E M W

Add \$5, \$2, \$7

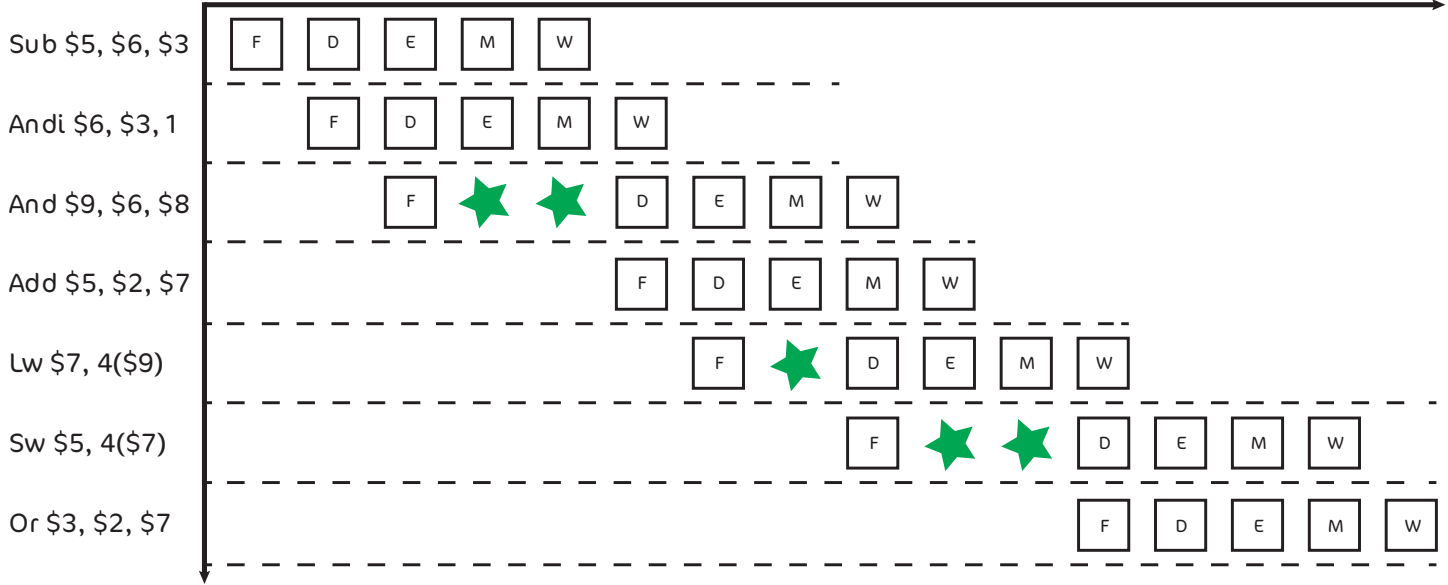
F D E M W

Lw \$7, 4(\$9)

F D E M W

- التعليمة السادسة: لدينا Stall في الدور 10 و11 لانتظار القيمة الصحيحة للسجل الخامس حتى تتم قراءتها

- التعليمة السابعة: لا يوجد فيها Stall



$$CPI = \frac{[5 + (IC - 1) + Stalls]}{IC} = \frac{[5 + 6 + 5]}{7} = \frac{16}{7} = 2.3$$

ملاحظة:

لا يختلف عدد الفقاعات وقيمة CPI عند افتراض أن قراءة السجلات تتم فعلياً في المرحلة E
فلو قمنا بحل التمرين السابق باعتبار أن عملية قراءة السجلات تتم في المرحلة E سنصل إلى نفس الحل:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1. sub \$5,\$6,\$3	F	D	E	M	W												
2. andi \$6,\$3,1		F	D	E	M	W											
3. and \$9,\$6,\$8			F	D	*	*	E	M	W								
4. add \$5,\$2,\$7				F	*	*	D	E	M	W							
5. lw \$7,4(\$9)						F	D	*	E	M	W						
6. sw \$5,4(\$7)							F	*	D	*	*	E	M	W			
7. or \$3,\$2,\$7									F	*	*	D	E	M	W		

لا يمكن هنا إجراء W و E على نفس السجل في نفس الدور لأن خاصية النفاذ المزدوج تنص على إمكانية الكتابة على ملف السجلات في نصف الدور ثم القراءة منه في نصف الدور، ولا تنص على إمكانية إجراء E في نصف الدور!

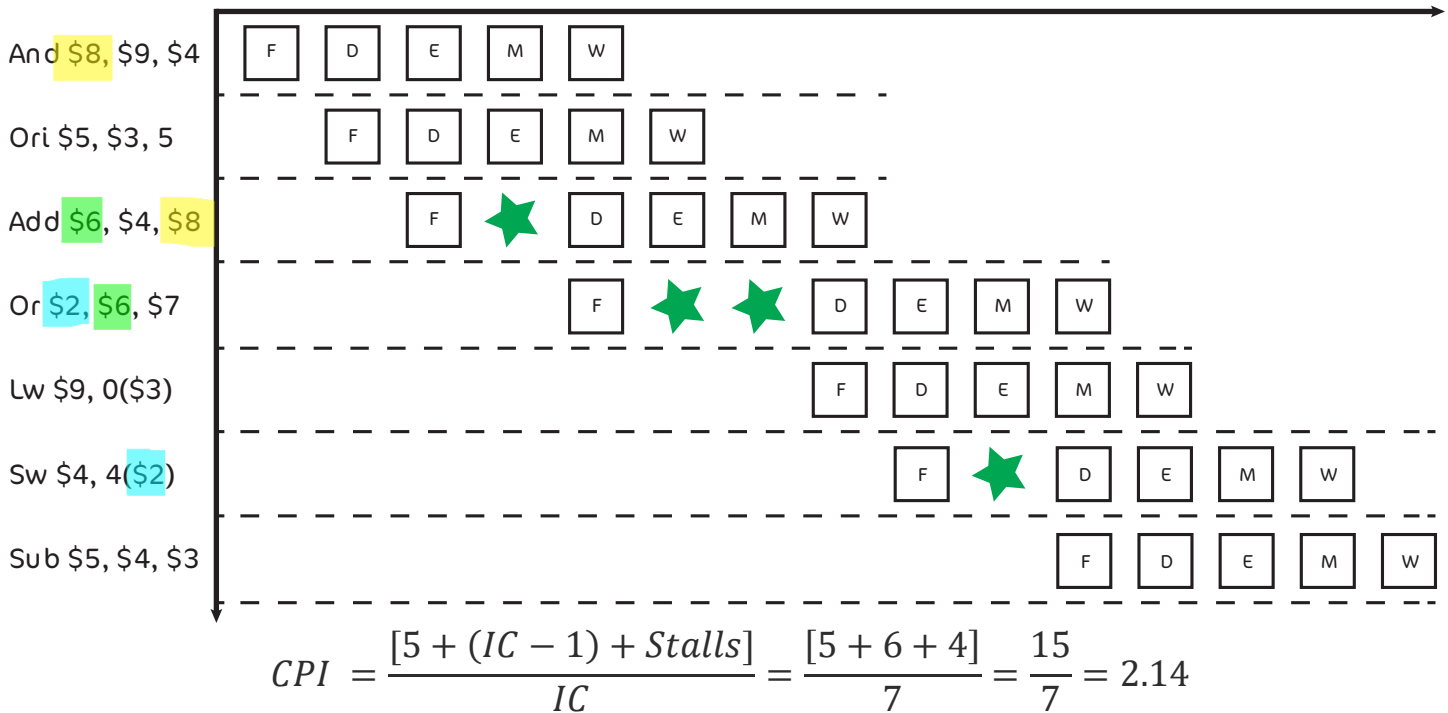
$$CPI = \frac{[5 + (IC - 1) + Stalls]}{IC} = \frac{[5 + 6 + 5]}{7} = \frac{16}{7} = 2.3$$

مثال للتدريب:

لدينا البرنامج التالي ينفذ على معالج MIPS متوارد، حدد ترابطات المعطيات الموجودة في البرنامج، ثم مثل أدوار التنفيذ باستخدام طريقة التأخير الزمني Stall (إضافة فقاعات Bubble) لمعالجة أعطال المعطيات بفرض عدم وجود أعطال بنيوية، وبفرض إمكانية النفاذ المزدوج لملف السجلات (إمكانية الكتابة والقراءة عليه بنفس الدور) (باعتبار أن قراءة السجلات تتم في المرحلة D)

التعليمات:

1. And \$8, \$9, \$4
2. Ori \$5, \$3, 5
3. Add \$6, \$4, \$8
4. Or \$2, \$6, \$7
5. Lw \$7, 0(\$3)
6. Sw \$4, 4(\$2)
7. Sub \$5, \$4, \$3

الحل:

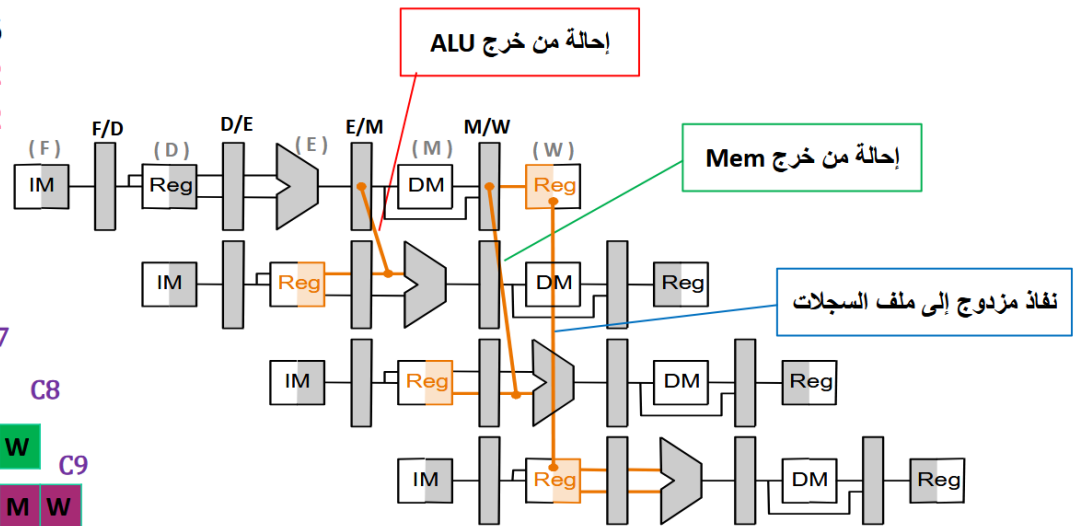
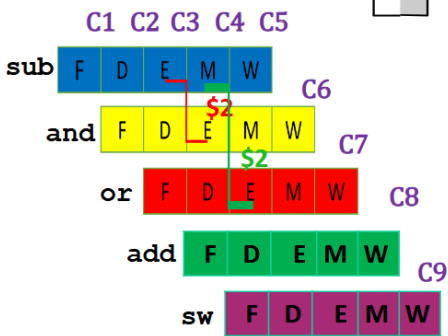
Don't give up,
your story isn't
over yet

حل أعطال المعطيات باستخدام دائرة الإحالة Forwarding Unit

تمرين:

لدينا معالج متوارد بخمس مراحل وفيه دائرة إحالة، وبفرض النفاذ المزدوج لملف السجلات، ما الإحالات الحاصلة عند تنفيذ ما يلي:

```
sub $2, $3, $1
and $12, $2, $5
or $13, $6, $2
add $14, $2, $2
sw $15, 10($2)
```



ملاحظة (1):

يمكن الإحالة من خرج ALU (من التعليمة الموجودة في المرحلة M) إلى دخل ALU, أو الإحالة من خرج M (من التعليمة الموجودة في المرحلة W) إلى دخل ALU

ملاحظة (2):

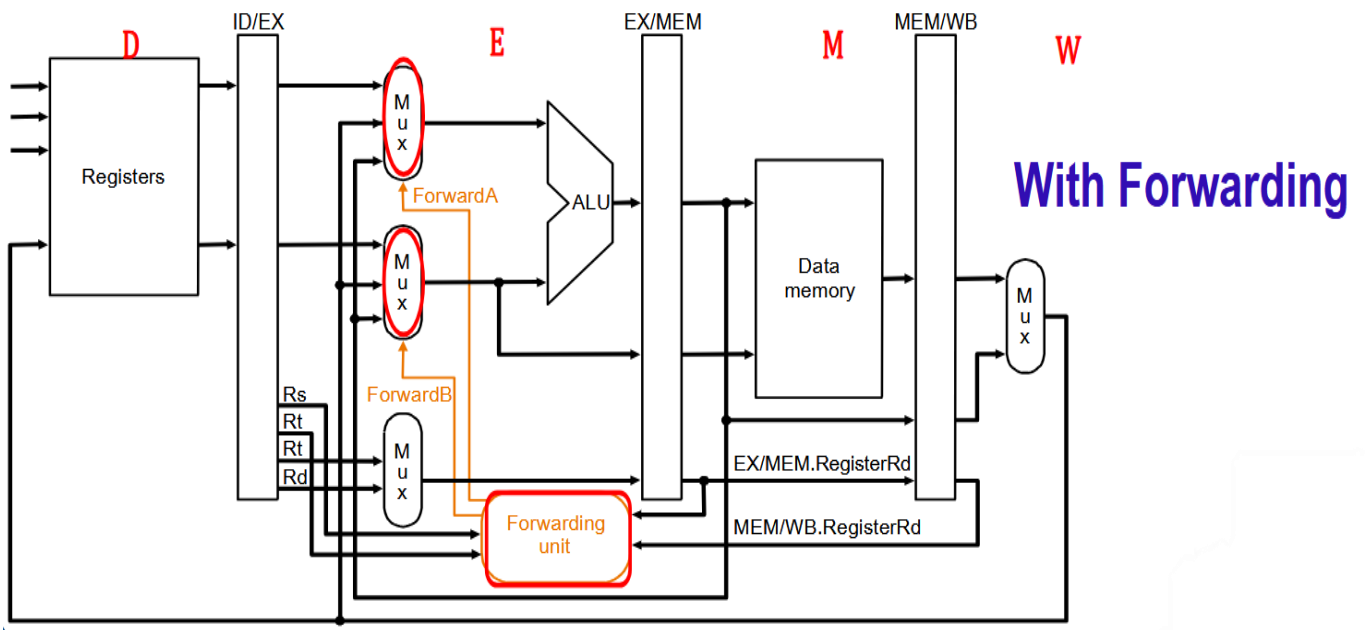
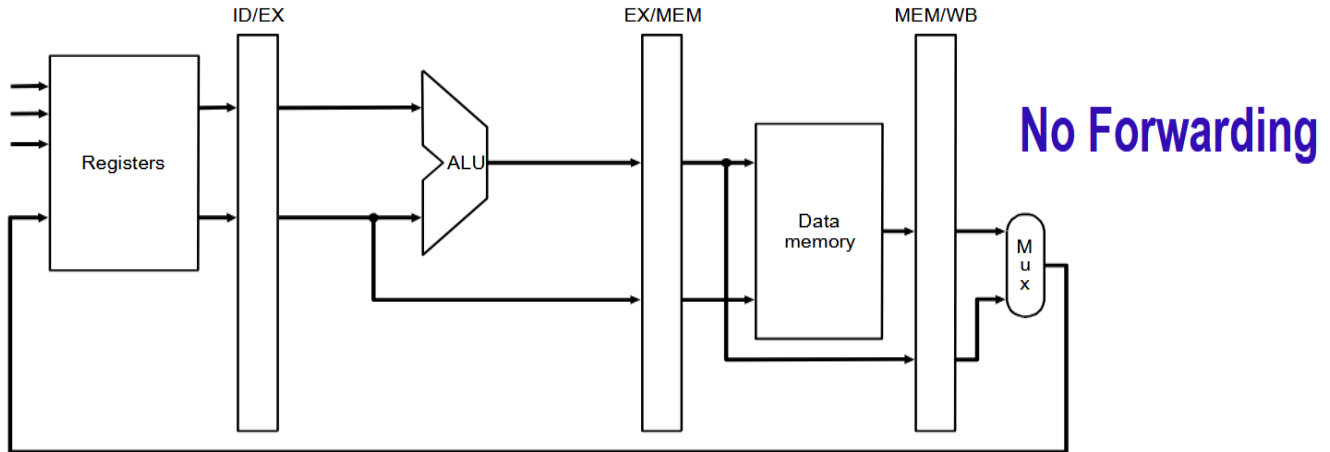
دائرة الإحالة حصراً بين المكونات في نفس الدور ولا يمكن وضع دائرة إحالة من الدور الثالث إلى الدور الرابع مثلاً



ولدارة الإحالة نوعين:

1. إما من خرج Memory -> إلى مدخل ALU

2. أو من خرج ALU -> إلى مدخل ALU



انتهت المحاضرة